

Distributed Background Job Scheduling Platform

A high-performance execution engine with real-time UI dashboard.

SUBMISSION INFORMATION

Registration Number:	RA2311026010746
Candidate Name:	Prudhvi
Project Platform:	Distributed Job Scheduler (Overdrive)
GitHub Repository:	https://github.com/prudhvi98491/distributed-job-scheduler
Date of Submission:	July 5, 2026

1. Executive Summary

This project implements a production-ready, highly reliable distributed background job scheduler system. Designed to handle concurrent task executions across multiple workers, the system guarantees fault tolerance, worker failover, and strict queue boundary limitations. The architecture includes features like multi-tenant organization workspaces, custom backoff retries, cron recurring tasks, and sequential workflows with dependency blocking. Users can monitor queues, track active nodes, and replay failed/DLQ tasks live via an Outfit-themed glassmorphic SPA dashboard.

Core Technology Stack

Component	Technology	Description / Implementation Details
API Framework	FastAPI / Uvicorn	Asynchronous endpoints, structured validation, OpenAPI docs
Database	SQLite (WAL Mode)	ACID storage with Write-Ahead Logging & busy timeout lock protection
Asynchronous ORM	SQLAlchemy + aiosqlite	Non-blocking connection pooling and entity relationship models
Recurring Cron	croniter	Expression parser that calculates dynamic schedules for active cron jobs
Dashboard SPA	Vanilla HTML5 / CSS3 / JS	Real-time state monitoring dashboard using modern glassmorphic design
Testing Engine	pytest + pytest-asyncio	Verification framework simulating job lifecycle and retries

2. Relational Database Design

The relational database schema is normalized to 3NF (Third Normal Form) to eliminate redundancy, safeguard referential integrity, and maximize performance. Tables are connected via primary and foreign key constraints, utilizing ON DELETE CASCADE or ON DELETE SET NULL configurations.

Table Name	Schema Purpose & Normalization Details
users	Stores credentials (hashed via bcrypt), roles (admin, user, viewer), and metadata.
organizations	Enables multi-tenancy, grouping projects and user access levels.
projects	Contains project workspaces owned by organizations.
queues	Defines job queues with priority, concurrency_limit, and retry policies.
jobs	Core queue table containing status (queued, running, completed, failed, dlq), payload, worker_id, and parent_job_id (workflows).
job_executions	Tracks execution details, duration (ms), worker, and trace errors.
workers	Worker fleet registry monitoring hostnames, IP addresses, heartbeats, and current loads.
cron_jobs	Schedules recurring cron jobs to dynamically queue new jobs on intervals.
dead_letter_queue	Holds permanently failed tasks for isolation and manual replay.

Concurrency Control and Atomic Claims

To ensure that jobs are claimed atomically and prevent duplicate execution (maintaining strict idempotency), the worker service executes the claiming block inside a database transaction with an IMMEDIATE lock. The SQL query retrieves a single eligible job from a queue that is active (not paused) and whose current running/claimed job count is strictly below the queue's configured concurrency_limit. This ensures horizontal scalability without race conditions.

3. REST API Endpoints

The backend exposes clean REST APIs with structured JSON validation, Pydantic parsing, and JWT-based security layers. The dashboard communicates with these endpoints to render statistics, dispatch tasks, pause/resume queues, and inspect logs.

Method	Endpoint	Description / Functionality
POST	/api/auth/register	Registers a new user account with hashed credentials.
POST	/api/auth/login	Authenticates credentials and issues a JWT token.
POST	/api/queues	Creates a queue specifying concurrency limit & priority.
PATCH	/api/queues/{id}	Updates queue configs or pauses/resumes queue state.
POST	/api/jobs	Enqueue single task (immediate, delayed, or workflow).
POST	/api/jobs/batch	Dispatches bulk job payloads under a single API call.
GET	/api/jobs	Lists jobs with pagination, queue, and status filters.
POST	/api/jobs/{id}/retry	Manually replays a failed or DLQ job.
GET	/api/metrics	Returns aggregate metrics, throughput trends, and error counts.

4. Resiliency & Test Verification

Worker Resiliency & Failover Recovery

Workers register themselves in the database and issue heartbeats every 5 seconds. If a worker goes offline for more than 15 seconds, it is marked as offline by healthy worker threads. Any running or claimed jobs on that node are automatically re-queued (status queued, worker_id reset) for failover recovery.

Automated Integration Testing

We implemented integration tests in tests/test_scheduler.py to verify relationships, retries, and sequential workflow unblocking. Running pytest completes successfully:

```
tests/test_scheduler.py::test_database_seeding_and_relations PASSED
tests/test_scheduler.py::test_job_execution_lifecycle_and_retries PASSED
tests/test_scheduler.py::test_workflow_dependencies_unblocking PASSED
===== 3 passed in 2.02s =====
```

Submission GitHub Repository

<https://github.com/prudhvi98491/distributed-job-scheduler>